

מבוא לתכנות מערכות – מבחן מועד א', סתיו תשע"ט

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

העמוד האחרון במבחן כולל חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

המכללה האקדמית תל חי, החוג למדעי המחשב

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 1 (15 נקודות)

א. מה עושה הפונקציה הבאה?

```
int que(const char* s1, const char* s2)
{
    int i;
    if(!(*s1))
        return 0;

    for(i=0; *(s2+i) && *(s1+i) && *(s1+i) == *(s2+i); i++)
        ;

    return que(s1+1, s2) + !*(s2+i);
}
```

הפונקציה מחזירה את מספר המופעים של s2 בתוך s1

ב. כתבו פונקציה בשם strip() שמורידה את כל ה-white spaces (רווחים, טאבים, ירידת שורה) מהתחלת ומסוף מחרוזת. על מנת לזהות white space השתמשו בפונקציה int isspace(char c) מהספרייה ctype.h שמקבלת תו c ומחזירה ערך שאינו 0 אם c הוא white space או 0 אחרת. בצעו את השינוי על הזכרון שבו נמצאת המחרוזת המקורית. אפשר להניח שמתקבלת מחרוזת לא ריקה. לדוגמא:

- עבור המחרוזת "foo bar" הפונקציה תשנה את המחרוזת ל-"foo bar"

מותר להשתמש בכל פונקציית ספרייה לבחירתכם.

```
void strip(char* s) {
    char *s1, *s2;

    for(s1=s; *s1!=0 && isspace(*s1); s1++)
        ;

    for(s2=s+strlen(s)-1; s2!=s1 && isspace(*s2); s2--)
        ;

    s2++;

    while(s1 != s2)
        *(s++) = *(s1++);

    *s = '\0';
}
```

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (25 נקודות)

בשאלה זו אתם צריכים לממש (חלקית) abstract data type של מבנה נתונים שנקרא multiset. מבנה הנתונים הזה הוא בעל המאפיינים הבאים:

- אפשר להכניס אליו ולהוציא ממנו אלמנטים
- לאלמנטים אין סדר מסויים (בניגוד למערך, למשל)
- אין הגבלה על מספר המופעים של כל אלמנט או על המספר הכולל של האלמנטים (בניגוד למערך)

א. כתבו ממשק (קובץ h) למבנה נתונים Multiset גנרי (שיכול לשמור אובייקטים מכל טיפוס שנבחר) אשר מאפשר את הפעולות הבאות:

- יצירת Multiset
- הריסת Multiset
- הוספת אלמנט ל-Multiset. יש ליצור עותק חדש של האלמנט שהועבר ולשמור אותו.
- הסרת אלמנט מה-Multiset. במידה וקיימים מספר מופעים של האלמנט הפונקציה תסיר אחד מהם.
- בדיקה האם אלמנט מסויים קיים ב-Multiset. הפונקציה תחזיר TRUE אם קיים לפחות מופע אחד של האלמנט או FALSE אחרת.
- החזרת מספר המופעים של אלמנט. למשל: אם ה-Multiset שומר מחרוזות ואנחנו מעבירים לפונקציה את המחרוזת "Hello" הפונקציה תחזיר את מספר המופעים של המחרוזת "Hello" (שווה למספר הפעמים שהמחרוזת הוכנסה)
- החזרת המספר הכולל של האלמנטים ב-Multiset

```
#ifndef MULTISSET H
```

```
#define MULTISSET H
```

```
typedef struct Multiset* Multiset;
```

```
typedef boid* Element;
```

```
Multiset MultisetCreate(bool (*is equal)(Element, Element),
```

```
Element (*cpy)(Element), void (*fre)(Element));
```

```
void MultisetDestroy(Multiset);
```

```
void MultisetAdd(Multiset, Element);
```

```
void MultisetRemove(Multiset, Element);
```

```
bool MultisetExists(Multiset, Element);
```

```
unsigned int MultisetNumElements(Multiset, Element);
```

```
unsigned int MultisetSize(Multiset);
```

```
#endif
```

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ה-Multiset שלנו ימומש ע"י רשימה מקושרת. עבור כל אלמנט חדש שנרצה להוסיף, נכניס אותו לראש הרשימה. בהוצאת אלמנט נעבור על הרשימה עד שנמצא את האלמנט אותו ברצוננו להוציא ונוציא אותו.

כתבו הגדרה ל-struct של Multiset וכן הגדרה ל-struct עבור רשימה מקושרת בו תשתמשו. ממשו את הפונקציה שיוצרת Multiset.

```
struct MultisetLL {  
    Element element;  
    struct MultisetLL* next;  
};  
typedef struct MultisetLL* MultisetLL;  
struct MultisetLL {  
    bool (*is equal)(Element, Element);  
    Element (*cpy)(Element);  
    void (*fre)(Element);  
    MultisetLL head;  
};  
Multiset MultisetCreate(bool (*is equal)(Element, Element),  
                        Element (*cpy)(Element), void (*fre)(Element)) {  
    Multiset ms = (Multiset)malloc(sizeof(struct Multiset));  
    ms->is equal = is equal;  
    ms->cpy = cpy;  
    ms->fre = fre;  
    ms->head = (MultisetLL)malloc(sizeof(struct MultisetLL));  
    ms->head->element = ms->head->next = NULL;  
    return ms;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את הפונקצייה שמחזירה את מספר המופעים של אלמנט מסויים.

```
unsigned int MultisetNumElements(Multiset m, Element e) {
```

```
    MultisetLL t = ms->head->next;
```

```
    int counter = 0;
```

```
    while(t != NULL)
```

```
        Counter += m->is equal(t->element, e);
```

```
    return counter;
```

```
}
```


המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (20 נקודות)

א. כתבו פונקציה שמקבלת שני מספרים a ו-b מטיפוס unsigned int ומחזירה את מספר הביטים שצריך לשנות כדי להפוך את a ל-b. למשל: עבור a=2 ו-b=3 צריך לשנות ביט אחד (מ-10 ל-11). כדי להפוך את 7 (111) ל-8 (1000) צריך לשנות 4 ביטים.

```
unsigned int change a2b(unsigned int a, unsigned int b) {
```

```
    int i=0, counter = 0;
```

```
    for(i=0; i<sizeof(unsigned int)*8; i++) {
```

```
        counter += (a & 1) ^ (b & 1);
```

```
        a >> 1;
```

```
        b >> 1;
```

```
    }
```

```
    return counter;
```

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו פונקציה בשם `print_hexa` שמקבלת מצביע מטיפוס `void*` למספר שלם וכן את מספר הבתים שהטיפוס תופס בזיכרון ומדפיסה את הייצוג ההקסהדצימלי (לפי בסיס 16) של המספר.
למשל:
עבור הקוד:

```
unsigned char c = 10;  
print_hexa((void*)(&c), sizeof(c));
```

הפונקציה `print_hexa` תדפיס

0A

עבור הקוד

```
unsigned int x = 257;  
print_hexa((void*)(&x), sizeof(x));
```

הפונקציה תדפיס (בהנחה ש-`sizeof(x)=4`)

00000101

לנוחיותכם, הפונקציה מכילה מערך עם כל הספרות של בסיס 16.

```
void print_hexa(void* p, int psize) {  
    char hex_digits[] = {'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F'};  
    int i;  
    for(i=psize-1; i>=0; i--) {  
        printf("%c", hex_digits[p[i] >> 4]);  
        printf("%c", hex_digits[p[i] & 0x0F]);  
    }  
}
```

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 4 (15 נקודות)

כתבו פונקציה שמקצה מערך דו-מימדי של char תוך שימוש בשתי קריאות malloc בלבד. הפונקציה תקבל את מספר השורות ומספר העמודות ותחזיר את כתובת המערך הדו-מימדי שהוקצה. לדוגמא: עבור הקריאה הבא לפונקציה:

```
char** p = allocate2D(100, 50);
```

ההשמה הבאה תהיה חוקית:

```
p[99][49] = 'A';
```

```
char** allocate2D(unsigned int ncols, unsigned int rows)
```

```
    char **p = (char*)malloc(ncols*sizeof(char));
```

```
    p[0] = (char)malloc(ncols*rows*sizeof(char));
```

```
    int i;
```

```
    for(i=1; i<ncols; i++)
```

```
        p[i] = p[0] + rows*i*sizeof(char);
```

```
    return p;
```

```
}
```

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for writing or drawing. There are no margins, text, or other markings present.

שאלה 5 (25 נקודות)

א. נתון הקובץ months.txt שמכיל את 12 השורות הבאות:

January
February
March
April
May
June
July
August
September
October
November
December

מה תדפסנה הפקודות הבאות:

```
egrep "^[OND].*" months.txt .1
```

October
November
December

```
egrep "[^ry]$" months.txt .2
```

March
April
June
August

```
head -n 6 months.txt | cut -b1-3 | egrep ".a." .3
```

Jan
Mar
May

```
sort months.txt | tail -n 1 .4
```

September

המכללה האקדמית תל חי, החוג למדעי המחשב

בסעיפים ב' ו-ג' תכתבו קוד שיעבוד על קובץ csv שבו יש ציוני סטודנטים בקורסים. בקובץ csv כל שורה מכילה מידע שמחולק לשדות ע"י פסיקים (התו ";"). הקובץ עליו תעבדו מכיל את המידע הבא:

- בשורה הראשונה יימצאו שמות הקורסים, למעט השדה הראשון שיכיל את המילה Student
- כל שורה אחרת תכיל ציונים של סטודנט מסוים. השדה הראשון יכיל את שם הסטודנט, שאר השדות מכילים את הציונים בקורסים שהשם שלהם נתון בשורה הראשונה.

למשל: בקובץ הבא:

```
Student,CPP,Java,Big Data
John,81,77,88
Jane,79,100,96
Jim,84,83,75
Jerry,76,80,79
```

- ישנם ציונים של ארבעה קורסים: CPP (שדה שני), Java (שדה שלישי) ו-Big Data (שדה רביעי).
- הסטודנט John קיבל 81 ב-CPP, 77 ב-Java ו-88 ב-Big Data
- הסטודנטית Jane קיבלה 79 ב-CPP, 100 ב-Java ו-96 ב-Big Data
- וכו'

ב. כתבו פונקציית bash בשם get_field שמקבלת שם של קובץ ושם של קורס ומוצאת את מספר העמודה של הקורס. במידה והקורס לא קיים הפונקציה תחזיר 1-. אתם יכולים (אבל לא חייבים, יש כמה דרכים לכתוב את הקוד המבוקש!) להשתמש בפקודה הבאה בשביל להפריד שדות לשורות נפרדות:

"\n", ",", "tr", כאשר הקלט לפקודה מגיע מה-standard input או משם של קובץ.

למשל: הפקודה

```
echo "One,Two,Three" | tr ",," "\n"
```

תדפיס

```
One
Two
Three
```

```
function get field {
    head -n1 $1 | tr ",," "\n" | grep -n $2 | cut -f1 -d":"
}
```


המכללה האקדמית תל חי, החוג למדעי המחשב

ג. כתבו סקריפט שמקבל שם של קובץ בפורמט המתואר למעלה ושם של קורס ומחזיר את הציון הממוצע בקורס. אפשר להתעלם מנקודה עשרונית – תוצאת החילוק ש-bash מחזיר היא טובה מספיק. במידה והקורס לא קיים החזירו הודעה מתאימה. מותר להשתמש בפונקציה מסעיף ב גם אם לא מימשתם אותה.
למשל: עבור הקובץ למעלה נקבל:

- הציון הממוצע ב-CPP הוא $(81+79+84+76)/4 = 320/4 = 80$
- הציון הממוצע ב-Java הוא $(77+100+83+76)/4 = 336/4 = 84$
- הציון הממוצע ב-Big Data הוא $(88+96+75+79)/4 = 338/4 = 84.5$ (למעשה התוצאה היא 84.5 אבל bash יעגל כלפי מטה)

```
if [[ $# -ne 2 ]] ; then
```

```
    echo -e "\nUsage: <csv-file> <column-name>\n"
```

```
    exit
```

```
fi
```

```
file=$1
```

```
field=$2
```

```
col=`get field $file $field`
```

```
if [[ $col == "" ]] ; then
```

```
    echo "Could not find $field in $file"
```

```
    exit
```

```
fi
```

```
t=0
```

```
n=0
```

```
for x in `tail -n +2 $file | cut -f $col -d ","` ; do
```

```
    t=$((t+x))
```

```
    n=$((n+1))
```

```
done
```

```
echo "The average grade is "$((t/n))
```

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

פקודות bash

- cat – מדפיסה תוכן של קובץ למסך
- head – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- tail – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- ls – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- sort – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- wc – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ
- read – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- find – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסויימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- grep – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v).
- egrep – זהה ל-grep -e
- cut – מחזירה שדות מסויימים בכל שורה (-f) בהתאם ל-delimiter (-d). ניתן גם להחזיר בתים מסויימים בכל שורה (-b)
- sed – עורכת טקסט הנתון כקלט. הפעולה s תשנה שורת טקסט.

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות ב-C

- int putchar(int character, FILE* fp) – כותבת את character ל-fp.
- int fgetc(FILE* fp) – קוראת תו אחד מתוך fp.
- scanf(const char* format,...) – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. fscanf(FILE* stream, const char* format,...) – אותו דבר כאשר הקלט נקרא מתוך stream.
- sscanf(const char* s, const char* format,...) – אותו הדבר כאשר הקלט נקרא מתוך s.
- int getline(char** lineptr, size_t* n, FILE* stream) – קוראת שורה לתוך lineptr, מקצה זכרון עבור lineptr אם אין מספיק (גודל ה-buffer הנוכחי מצוי ב-n). מחזירה את מס' התווים שקראה או -1 אם לא הצליחה לקרוא.
- printf(const char* format,...) – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- fprintf(FILE* stream, const char* format,...) – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- sprintf(const char* s, const char* format,...) – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- void* malloc(size_t size) – מקצה זכרון בגודל size
- void* calloc(size_t num, size_t size) – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- void* realloc(void* ptr, size_t size) – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.
- const char* strstr(const char* str1, const char* str2) – מחזירה את המקום שבו מופיעה str2 ב-str1 (או NULL אם היא לא מופיעה)
- int strcmp(const char* str1, const char* str2) – משווה את str1 ל-str2, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם str1 קטן לקסיקוגרפית מ-str2 או ערך גדול מ-0 אחרת.

המכללה האקדמית תל חי, החוג למדעי המחשב

- `size_t strlen(const char* str)` – מחזירה אורך של מחרוזת.
- `int fseek(FILE *fptr, long offset, int origin)` - מזיזה את `fptr` ל-`offset` בתיים ביחס ל-`origin`. המשתנה `origin` יכול להיות
 - `SEEK_SET` – תחילת הקובץ
 - `SEEK_CUR` – מיקום נוכחי
 - `SEEK_END` – סוף הקובץ

הפונקציה תחזיר 0 בהצלחה או ערך שונה מ-0 אחרת.

Regular expressions

.	Match any character
*	Match 0 or more time the preceding character
+	Match 1 or more time the preceding character
?	Match 0 or 1 times the preceding character
{n}	Match exactly n of the preceding item
{n, m}	Match between n and m of the preceding item
{n,}	Match n or more of the preceding item
^	Beginning of line
\$	End of line
[]	Match a range of characters
[^]	Match a character which is not contained in the range
	Or – match either the pattern before or after the
()	Group together parts of the search expression